

Planning Document — End-to-End Encryption (E2EE) for Audio/Video via Web Client (Vanilla Chrome) + SFU (Str0m)

Scope & Goals

- **Client:** Vanilla Chrome web application (JavaScript).
 - **Server:** SFU implemented using **Str0m** (Rust).
 - **Goal:** Add E2EE such that **cloud SFU is “zero trust” for media content** (cannot decrypt audio/video), while retaining SFU benefits (NAT traversal, bandwidth adaptation, forwarding, scalability).
 - **Non-goals (initial):** Server-side recording, server-side transcription, media inspection, content-based routing—these typically conflict with strict E2EE unless using special trust models.
-

1) Current Status & Technical Analysis

1.1 What “E2EE” Means in WebRTC/SRTP Systems

Most WebRTC deployments today use **hop-by-hop encryption**:

- Client ↔ SFU: DTLS-SRTP establishes SRTP keys; SFU decrypts and re-encrypts when forwarding.
- This protects media on the network, but **not** from the SFU operator.

True E2EE for conferences means:

- Media is encrypted **at the sender endpoint** with keys **unknown to SFU**, and decrypted only at receiving endpoints.
- The SFU may still perform transport security (DTLS-SRTP) but only sees **ciphertext media payload**.

Typical tradeoffs in E2EE conferencing (industry-wide):

- Reduced server-side features (recording, PSTN/SIP interop, server mixing, transcription).
- Harder debugging/analytics because SFU can’t inspect payload.

1.2 Zero Trust Policy for Cloud Media

Adopt a “zero trust relay” stance:

- Assume SFU and cloud infrastructure are **untrusted for confidentiality**.
- SFU can be trusted only for:
 - packet forwarding
 - congestion control assistance
 - RTCP feedback routing
 - bandwidth adaptation and layer selection (when metadata is available)
- SFU must **not** have access to end-to-end media keys.

Threat model examples:

- Cloud operator compromise
- Insider access
- VM snapshot / memory scraping
- Logging of decrypted frames

Target security statement:

“Compromise of SFU infrastructure does not reveal media plaintext, only traffic metadata.”

1.3 RFC / Standards & Related Work (Key References to Track)

You will want a reference section (to cite in design docs / security review). Core standards to consider:

- **RTP**: RFC 3550 (packet format, header fields, RTCP)
- **SRTP**: RFC 3711 (encryption/authentication of RTP payload)
- **DTLS-SRTP**: RFC 5764 (keying SRTP via DTLS in WebRTC)
- **SDP security descriptions (SDS)**: RFC 4568 (keying SRTP via SDP; generally not recommended vs DTLS)
- **RTP header extensions**: RFC 8285
- **Encrypted RTP header extensions** (“cryptex” direction): RFC 6904 (mechanism for encrypting RTP header extensions with SRTP context)
- **ZRTP** (in-band SRTP key agreement; relevant if you pursue media-channel-only keying): RFC 6189

Additionally (not RFCs in all cases), monitor:

- WebRTC security architecture RFCs (IETF RTCWEB security documents)
- IETF SFrame (frame-level E2EE) work, and W3C Insertable Streams API evolution

Practical note: In Chrome today, E2EE is most commonly implemented with Insertable Streams + frame encryption (SFrame-like), while SRTP remains hop-by-hop for transport.

1.4 Current SFU Implementation Landscape (High-Level)

Most SFUs operate on RTP/RTCP without decoding:

- They route streams by SSRC/MID/RID
- They forward RTCP feedback (PLI/FIR/NACK/TWCC)
- They can do simulcast/SVC layer selection, often based on RTP header extensions.

In E2EE mode, SFU can still do most of this **if** required routing/adaptation metadata is available outside the encrypted payload.

1.5 RTP Header Handling: What Stays Visible vs Protected

RTP fixed header fields (in clear even with SRTP):

- version, padding, extension bit, CC
- marker, payload type
- sequence number
- timestamp
- SSRC
- CSRC list (if used)

SRTP protects:

- Payload (encrypted)
- Optional authentication tag (integrity)
- Optional header-extension encryption if RFC 6904 is used

Implication: Even with SRTP and E2EE, an observer/SFU can still learn:

- packet rate and timing (traffic analysis)
- SSRC correlations (stream identity)
- codec/pt usage (sometimes payload type mapping leaks)
- speaking patterns / video activity (via packet cadence)

This is **metadata leakage**; E2EE usually protects content but not all metadata.

1.6 Why “Cryptex” / Encrypted Header Extensions Matter

SFUs rely on RTP header extensions for:

- transport-wide congestion control (TWCC)
- absolute send time
- MID/RID (routing)
- audio level / voice activity indicators
- dependency descriptors for scalable video layers

But these extensions can leak:

- who is talking
- frame boundaries / keyframe markers

- bandwidth estimation hints
- per-stream identifiers

Cryptex approach (direction):

- Encrypt some header extensions end-to-end
- Keep only minimal forwarding-required IDs visible
- Possibly allow SFU to read a *subset* of metadata (policy choice)

1.7 How SRTP is Used for E2EE Systems

A robust modern design uses **two layers**:

1. **Outer hop-by-hop SRTP (DTLS-SRTP):**
 - Client ↔ SFU transport confidentiality/integrity.
 - Required for WebRTC baseline interoperability.
1. **Inner end-to-end media encryption (frame/payload level):**
 - Encrypt encoded frames before packetization (or encrypt payload data after encoding but before send).
 - SFU forwards ciphertext.

This “double encryption” model is common because:

- WebRTC stack expects SRTP on the wire.
- E2EE keys must not terminate at SFU.

1.8 Key Exchange Options (Including “Media-Channel Only”)

You requested: “**keys exchanged completely through the media channel.**” Options:

Option A — DataChannel-based E2EE control (recommended for Vanilla Chrome)

- Use WebRTC DataChannel to exchange E2EE keying messages.
- Requirement: SFU must support **datachannel forwarding/proxying** (so it doesn’t terminate the keys).
- Pros: Works well with browsers; flexible; can implement MLS-like group keying on top.
- Cons: Requires SFU changes (SCTP/DTLS data channel proxy).

Option B — In-band RTP keying (ZRTP-like)

- Exchange key agreement messages as special RTP payloads (or RTP header extension messages).
- Pros: “Media channel only” in the strictest sense.
- Cons: Not natively supported in Chrome WebRTC; you end up implementing a parallel keying protocol and careful anti-downgrade protection.

Option C — Use signaling (most common in practice)

- Exchange keys via HTTPS signaling protected by authentication/identity.
- Pros: Simplest to implement.
- Cons: Not “media-channel only,” and increases trust in signaling path.

A pragmatic plan is: **A first**, with B as a fallback research track.

1.9 Example: Scalable H.264 Video (Simulcast / SVC) Under E2EE

Challenge: SFU needs to adapt bandwidth by dropping layers, but layers’ structure is inside encrypted payload.

Approach 1: Simulcast (preferred)

- Send multiple encodes: low/medium/high
- Each layer uses distinct SSRC and RID/MID mapping (visible metadata)
- SFU can forward only the chosen layer **without decrypting payload**
- Sender encrypts each encoded frame once per layer stream (still manageable)

Approach 2: True SVC in one stream

- Single SSRC, multiple spatial/temporal layers in payload
- SFU needs dependency info to drop enhancement layers safely
- Solution: provide **cleartext dependency metadata** (e.g., a header extension) while encrypting frame data
- Security tradeoff: leaks structure (keyframe cadence, temporal layering)

Recommended for H.264 in Chrome today: Simulcast.

It aligns with SFU operation and minimizes metadata requirements.

2) Str0m Library Examination & SFU-Side Integration Strategy

2.1 Where E2EE Hooks Into an SFU

A Str0m-based SFU typically has these stages:

ICE/DTLS -> SRTP unwrap -> RTP demux (SSRC/MID/RID) -> routing logic -> SRTP wrap -> send

|-> RTCP handling (NACK/PLI/TWCC/REMB)

For E2EE you want:

- SFU may still terminate **DTLS-SRTP** (outer layer) for WebRTC compliance.
- But SFU must treat media payload as **opaque bytes** (ciphertext) and never decode.
- The inner E2EE encryption/decryption happens only at endpoints.

2.2 Minimal SFU Changes Required for E2EE

Must-have:

1. **Preserve ciphertext payload end-to-end:**
 - Forward RTP packets (post outer-SRTP decryption) without modifying encrypted media content bytes (except as required for routing).
2. **Don't rewrite payload-dependent fields:**
 - Avoid transformations that require parsing codec payload (no VP8/VP9/H264 payload inspection).
3. **Retain routing metadata:**
 - MID/RID/SSRC must remain usable.
4. **RTCP feedback routing still works:**
 - NACK/PLI/FIR/TWCC must flow; SFU may originate some RTCP (e.g., RR) without seeing plaintext.

Nice-to-have:

- Support for **encrypted header extensions** (RFC 6904 style) when you want to hide more metadata, while still allowing required ones in the clear.

2.3 Supporting Many Clients: Do We Re-Encrypt Per Receiver?

This is a central scaling decision.

Model 1 — Group key / per-sender key (scales best):

- Each sender encrypts once per frame using a per-sender key.
- All receivers have the key(s) needed to decrypt.
- SFU just forwards.
- Encryption cost: $O(1)$ per frame (per layer stream).
- Key distribution cost: $O(N)$ per join/rekey.

Model 2 — Pairwise keys (stronger compartmentalization, worse scaling):

- Sender encrypts separately for each receiver.
- Encryption cost: $O(N)$ per frame per sender (not great).
- Useful if you want “recipient privacy” vs other participants, uncommon in conferencing.

Recommendation: Start with **Model 1 (per-sender keys, group-managed)** plus periodic rekey/ratcheting.

2.4 SFU DataChannel Proxy for “Media-Channel Only” Key Exchange

If you want key exchange through the media channel:

- Implement SFU as a **DataChannel forwarder** rather than terminating SCTP/DTLS.

- Two patterns:
 - a. **SCTP association proxy** (harder): SFU relays SCTP chunks.
 - b. **App-level data relay** (more common): SFU terminates DTLS for transport, but relays **end-to-end encrypted key messages** on an SFU-controlled data path.
 - Still “media-channel” in the sense it uses the same ICE/DTLS transport.
 - But keys are encrypted at application layer so SFU can’t read them.

For strictest E2EE semantics, ensure:

- Anti-downgrade signals (participants detect if E2EE control is missing)
 - Transcript of group membership and key epochs is authenticated end-to-end
-

3) Client (Vanilla Chrome): Encryption Implementation

3.1 Insertable Streams Feasibility

Yes—**Insertable Streams** are the most practical path in Vanilla Chrome:

- You can transform encoded audio/video frames in JavaScript:
 - Encrypt outgoing RTCEncodedAudioFrame/RTCEncodedVideoFrame
 - Decrypt incoming frames before decode/render

Placement: “Encoded transform” layer:

Camera/Mic -> Encoder -> (E2EE Encrypt Transform) -> SRTP -> network -> SRTP -> (E2EE Decrypt Transform) -> Decoder -> Render

This keeps:

- WebRTC transport stack unchanged
- SFU unaware of media plaintext

3.2 Keys Exchange “Completely Through Media Channel”

Pragmatic client plan:

- Use a **DataChannel** for E2EE control messages (join, rekey, epoch, sender key announcements).
- Encrypt those control messages end-to-end (so SFU only relays ciphertext).
- If SFU can’t forward DataChannel end-to-end, you can:
 - Send E2EE control messages via **RTP side-channel** (experimental track), or
 - Use signaling initially (fallback).

3.3 DTLS End-to-End Consideration

DTLS in WebRTC typically terminates at each peer connection end:

- In SFU architecture, DTLS is **client↔SFU**, not client↔client.
- Achieving DTLS “end-to-end” across SFU would require the SFU to behave like a transparent relay for DTLS records—this is **non-trivial** in multi-party.

Recommendation: Keep DTLS-SRTP hop-by-hop for transport security, and provide E2EE at frame level.

3.4 Custom Chromium Build: When/Why

You mentioned possibly shipping a custom Chromium build to:

- Reuse Chromium’s crypto primitives and DTLS engine
- Implement deeper integration (e.g., encrypted header extensions with policy control)
- Improve performance vs JS transforms

Reality check:

- Custom Chrome distribution is heavy operationally.
- Insertable Streams is usually good enough for initial E2EE, especially if you:
 - Use WebCrypto SubtleCrypto (AES-GCM) and optimize buffer handling
 - Minimize per-frame allocations

Phased approach:

1. Prototype with Insertable Streams (fast iteration).
2. Only move to custom Chromium if performance/feature gaps are proven.

4) Detailed Implementation Plan (SFU + Web Clients)

4.1 Architecture Schema

```

Web Client A (Chrome + JS)          SFU (Str0m)          Web Client
C (Chrome + JS)-----
-----Encode
terminate      SRTP (DTLS)E2EE Encrypt (inner)          -->      Route
RTP/RTCP metadata      E2EE Decrypt (inner)SRTP Protect (outer; DTLS-SRTP)  --
>      Forward opaque bytes          Decode / RenderWeb Client B (Chrome + JS)
Web Client D (Chrome + JS)-----
-----Encode
SRTP (DTLS)E2EE Encrypt (inner)          -->
E2EE Decrypt (inner)SRTP Protect (outer; DTLS-SRTP)  -->
Decode / RenderNotes:- Outer DTLS-SRTP is hop-by-hop on each Client <-> SFU
leg.- Inner E2EE stays end-to-end; SFU does not have E2EE keys and cannot
decrypt media.
```


4.2 Step Plan — Server (Rust / Str0m)

S0 — Baseline SFU capability validation

- Confirm Str0m SFU can:
 - accept multiple PeerConnections
 - forward RTP/RTCP between them
 - support simulcast signaling (RID/MID) and layer selection policy
- Output: test harness + call topology tests

S1 — Define E2EE policy + metadata contract

- Decide what SFU needs visible:
 - MID/RID/SSRC mappings
 - TWCC extension (if used)
 - Minimal layer selection metadata (prefer simulcast)
- Decide what should be hidden:
 - audio level, voice activity, dependency descriptors (optional)
- Output: “metadata allowedlist” spec

S2 — Opaque payload forwarding

- Ensure SFU does not parse codec payload.
- Verify SFU does not rewrite frame data.
- Output: “payload integrity invariants” + tests (byte-for-byte payload equality after forward)

S3 — Rekey-safe routing

- Ensure SFU can handle:
 - key epoch changes (payload changes but headers stable)
 - no reliance on payload signatures
- Output: rekey test plan

S4 — E2EE control channel forwarding

- Implement one of:
 - DataChannel forwarding/proxying, OR
 - Application-level encrypted control messages via SFU relay
- Output: control plane message routing + membership awareness (who gets which messages)

S5 — Observability (E2EE-safe)

- Add metrics that do not require plaintext:
 - packet loss, jitter, bitrate, RTT
 - per-SSRC forwarding stats

- Output: logging policy and metrics exporters

4.3 Step Plan — Client (JS, optional C++)

C0 — Insertable Streams prototype

- Implement:
 - outgoing encrypt transform
 - incoming decrypt transform
- Choose crypto:
 - AES-GCM with per-frame nonce derived from (SSRC, seq, epoch) or a counter
 - include KeyID/Epoch in a small unencrypted frame header inside payload
- Output: working 1:1 call E2EE in Chrome

C1 — Group key management

- Implement per-sender keys:
 - each participant has sender key for outgoing frames
 - distribute sender key to group via E2EE control channel
- Output: multi-party E2EE conference demo (SFU in the middle)

C2 — Rekey/ratchet

- Implement:
 - rekey on membership change (join/leave)
 - periodic rekey (time or packet count)
 - anti-replay window
- Output: security regression tests

C3 — Scalability plan for H.264

- Prefer simulcast:
 - encrypt each simulcast layer stream independently (same key, different SSRC/rid)
 - SFU selects layer by SSRC/RID only
- Output: adaptive forwarding demo under bandwidth changes

C4 — UX indicators

- Show user-facing E2EE state:
 - “E2EE ON” when keys established for all participants
 - detect downgrade (missing transforms / missing key epoch)
- Output: UI + telemetry events

4.4 Dependencies Between Steps (High-Level)

S0 -> S1 -> S2 -> (S4) -> S5

C0 -> C1 -> C2 -> C4

-> C3

Critical cross-dependency:

C1/C2 require S4 (or alternate key exchange path).

C3 depends on S1 metadata contract and S0 simulcast support.

5) Conclusions & Future Development Possibilities

5.1 Short System Analysis

Security level increase vs classic SFU:

- Classic SFU: cloud can decrypt media → compromise reveals content.
- E2EE SFU: cloud sees ciphertext only → compromise reveals **metadata + traffic patterns** but not content.

Residual risks:

- Metadata leakage from RTP headers and necessary extensions
- Endpoint compromise (malware, XSS) defeats E2EE
- Key management bugs (replay, nonce reuse, weak rekey)

5.2 Comparison with Existing E2EE Systems (Conceptual)

E2EE conferencing systems in the market commonly:

- Use frame-level encryption at endpoints
- Maintain hop-by-hop SRTP for transport
- Restrict server-side features
- Provide UI attestation (E2EE indicators)

Your proposed approach aligns with this mainstream model.

5.3 Future Enhancements

1. **Encrypted header extensions (Cryptex direction)**
 - Encrypt more metadata while preserving SFU essentials
 - Explore which extensions can be safely hidden without breaking CC/routing
2. **Stronger group keying (MLS-style)**
 - Formalize group membership, authentication, and forward secrecy
 - Better post-compromise security
3. **Hardware-backed keys**
 - WebCrypto + platform secure enclaves where possible

- Reduce key exfiltration risk from JS heap
 - 4. **Optional trust tiers**
 - “Strict E2EE” mode: no server features
 - “Enhanced features” mode: allow limited trusted service for recording, etc.
-

One concise follow-up (so I can tailor this immediately)

Do you want **strict “media-channel only” key exchange** (no HTTPS signaling whatsoever), or is “keys delivered through an E2EE data channel over the same ICE/DTLS transport” acceptable?

If you answer that, I can lock the design into **one keying model** and produce concrete message formats (KeyID/epoch/nonce layout), plus a Str0m-centric module breakdown you can directly turn into tasks.